```
    console.error(error);
  } finally {
    await closeBrowser();}})();
```

Now that the page has the value set, you need to call *evaluate()* once more to scrape the search results. The function you pass to *evaluate()*, which is called the *pageFunction* parameter, can return a value. For example, the *getUrls()* function in the script shown below returns the contents of all the tags on the Google search results.

```
const { openBrowser, goto, evaluate,
closeBrowser } = require('taiko');
(async () => {
 try {
    await openBrowser();
    await goto('google.com');
    // Type "JavaScript" into the search
bar
    await evaluate(() => {
      document.querySelector('input[name
="q"]').value = 'Gauge Taiko';
    });

    await evaluate(() => {
      document.querySelector('input[valu
e="Google Search"]').click();
    });

    // Get all the search result URLs
    const links = await
evaluate(function getUrls() {
      return Array.from(document.
querySelectorAll('a cite').values()).
map(
        el => el.innerHTML
      );
    });
    console.log(links);
  } catch (error) {
    console.error(error);
  } finally {
    await closeBrowser();
  }
})();
```

Create your own superpowers

Taiko's features can be extended via plugins, which can allow users to take advantage of the *ChromeDevtoolsProtocol* with core Taiko concentrating on functionalities around UI automation tests.

One such feature in Chrome DevTools is the performance and the audit protocol for helping developers and testers monitor an application's performance. This is a good example of a plugin that lets users perform diagnostic actions on the website.

How Taiko plugins work

- Plugin names should have the prefix 'taiko-'; for example, taiko-taiko-diagnostics, taiko-screencast.
- Plugins should not have Taiko as a dependency; they should use the same instance as the users' script.
- Plugins should implement the *init()* method, which will be called by Taiko on load passing and event handler instances.
- Taiko communicates with plugins via events.

For a full list of features and APIs, do check the documentation.

Taiko's concise API with features like implicit waits and smart selectors make it an ideal choice for creating reliable UI tests. Its use of the Chrome DevTools protocol and its plugin architecture lets users add more superpowers, depending on what their applications need.

For a complete list of features and the full API, please go to *taiko.dev*. END

References

[1] *https://docs.taiko.dev*
[2] *https://taiko.dev*
[3] *https://github.com/saikrishna321/taiko-eyes*
[4] *https://github.com/saikrishna321/taiko-diagnostics*

Acknowledgement

I would like to thank Lubaina Rangwala, product marketing specialist (Taiko), and Zabil Cheriya Maliackal, product owner/lead developer (Taiko), for their inputs.

By: Sai Krishna

The author works as a lead consultant at ThoughtWorks. He has worked extensively on testing different mobile applications and building automation frameworks. He is an active contributor to Appium, Taiko, ATD, Taiko-Plugins, Selenium and is also a member of Appium org. He loves to contribute to open source technologies.